

CINDY HSU

Garden Grove, CA | twncindy@yahoo.com | GitHub | LinkedIn | Portfolio

PROFESSIONAL SUMMARY

GenAI / LLM Application Engineer with hands-on, production-grade experience building retrieval-augmented generation (RAG) systems — from hand-rolled primitives through a fully migrated, deployed, LangGraph-orchestrated, stateful multi-turn architecture live on Google Cloud Run. Designed a two-stage retrieval pipeline (ChromaDB + HuggingFace cross-encoder reranking), a provider-agnostic generation layer supporting OpenAI and Gemini with tiered failover, and a Corrective RAG (CRAG) fallback that grounds answers in live web search when local context and trained knowledge both fall short.

Brings 12+ years of production software engineering discipline — version control, testing, deployment, incident debugging — to a deliberate, focused transition into applied GenAI/LLM engineering. Comfortable working across the full stack of a modern AI application: orchestration frameworks, vector retrieval, multi-provider LLM integration, evaluation, and the operational realities of deploying and debugging systems once they're live.

Career note: 2019–2025, extended personal sabbatical (travel); 2025–present, self-directed transition into applied AI/ML engineering.

CORE SKILLS

Languages & Frameworks:	Python, FastAPI, LangGraph, LangChain (langchain-openai, langchain-google-genai)
RAG & Retrieval:	RAG pipeline design, ChromaDB, FAISS (early-stage prototyping), HuggingFace (sentence-transformers, cross-encoder reranking), two-stage retrieval architectures, embedding model migration
LLM Integration:	OpenAI API, Gemini API, multi-provider integration & failover, prompt engineering, MCP (Model Context Protocol) server development, Claude Desktop / Claude API
Engineering Practices:	retry logic & exponential backoff, async/await, pytest (unit, integration, and mocked-provider testing), Docker, Git & GitHub (branching, merge conflict resolution), REST APIs, environment variable & secrets management
Deployment & Tooling:	cloud deployment (Render, Google Cloud Run), AI-assisted development (Cursor, Claude), uv (Python dependency management), JSON, command line

PROJECTS

AI Chatbot with RAG — LangGraph Edition

2026 – Present

Tech Stack: Python · FastAPI · LangGraph · LangChain · ChromaDB · HuggingFace · OpenAI API · Gemini API · Tavily API · Docker · pytest · Cursor

Code: <https://github.com/cindyhsugit/ai-chatbot-fastapi-with-rag>

- Migrated orchestration to LangGraph and deployed live on Google Cloud Run; the original manual implementation remains live on Render as a deliberate before/after architectural comparison.
- Designed a two-stage retrieval architecture: wide-recall vector retrieval (top 20 candidates) followed by HuggingFace cross-encoder reranking (ms-marco-MiniLM-L6-v2) narrowed to the top 3, improving answer relevance beyond raw vector similarity alone.
- Modeled the pipeline as a typed StateGraph with conditional routing between context-grounded, trained-knowledge, and web-search generation paths, adding a MemorySaver checkpoint with a message-history reducer for persistent multi-turn conversational memory.
- Migrated vector storage from FAISS to ChromaDB and embeddings from OpenAI to a self-hosted HuggingFace model (all-MiniLM-L6-v2), removing external cost; resolved a vector-dimensionality mismatch (1536 vs. 384) across code paths.
- Built a provider-agnostic generation layer (OpenAI primary, Gemini fallback) with a knowledge-coverage failover triggering live web search (Tavily API) when local context and training data fall short, explicitly flagging web-sourced answers.
- Diagnosed a cross-provider response-format bug (Gemini structured blocks vs. OpenAI strings) and wrote a normalization layer; root-caused an intermittent memory failure to sampling variance at a decision boundary rather than a state bug.
- Maintained a pytest suite with 100% line coverage (763 statements, 70 tests) including mocked provider calls, graph-node unit tests, and end-to-end integration tests; verified retrieval correctness via adversarial testing.

Tech Stack: Python · FastMCP · httpx · GeoNames API · Claude Desktop · uv · MediaWiki API

Code: <https://github.com/cindyhsugit/mcp-server-simpsons-cities>

- Built and debugged an MCP server end-to-end against Claude Desktop exposing a live tool to find U.S. cities near a zip code sharing a name with entities from a popular TV show using FastMCP and async httpx.
- Integrated an authenticated geospatial API (GeoNames) and built a standalone data-collection script paginating MediaWiki category APIs to build a clean, versioned reference dataset.
- Diagnosed real integration hurdles: SDK breaking changes (FastMCP v2 rename, pinned to v1.x), HTTPS certificate mismatches, and multi-minute propagation delays; used MCP Inspector and structured stderr logging for deep traceability.
- Configured and tested as a local Claude Desktop integration (claude_desktop_config.json via uv), validating natural-language tool routing and correct responses.

PRIOR PROFESSIONAL EXPERIENCE — SOFTWARE ENGINEERING

Senior iOS Developer — StubHub, San Francisco, CA

March 2017 – May 2019

- Developed new features for different iOS versions, share extensions, widgets, and public/private libraries.
- Enhanced UI behavior for video/audio call states and responsive settings across iPad and iPhone.
- Improved localization support, increasing non-English speaking users by 300%.

Senior iOS Developer — Udemy, San Francisco, CA

April 2015 – February 2017

- Handled build and release processes while collaborating with QA overseas.
- Translated the codebase to Swift and increased Swift usage across the project.

Senior iOS Developer — Mercari, San Francisco, CA

September 2014 – April 2015

- Worked with the Japan team to maintain code consistency and support new feature development.
- Tested U.S. and Japan app versions and supported localization testing in English and Japanese.

EDUCATION

M.S. Computer Science — University of Southern California, 2005 – 2007

B.S. Computer Science — Iowa State University, 2001 – 2005